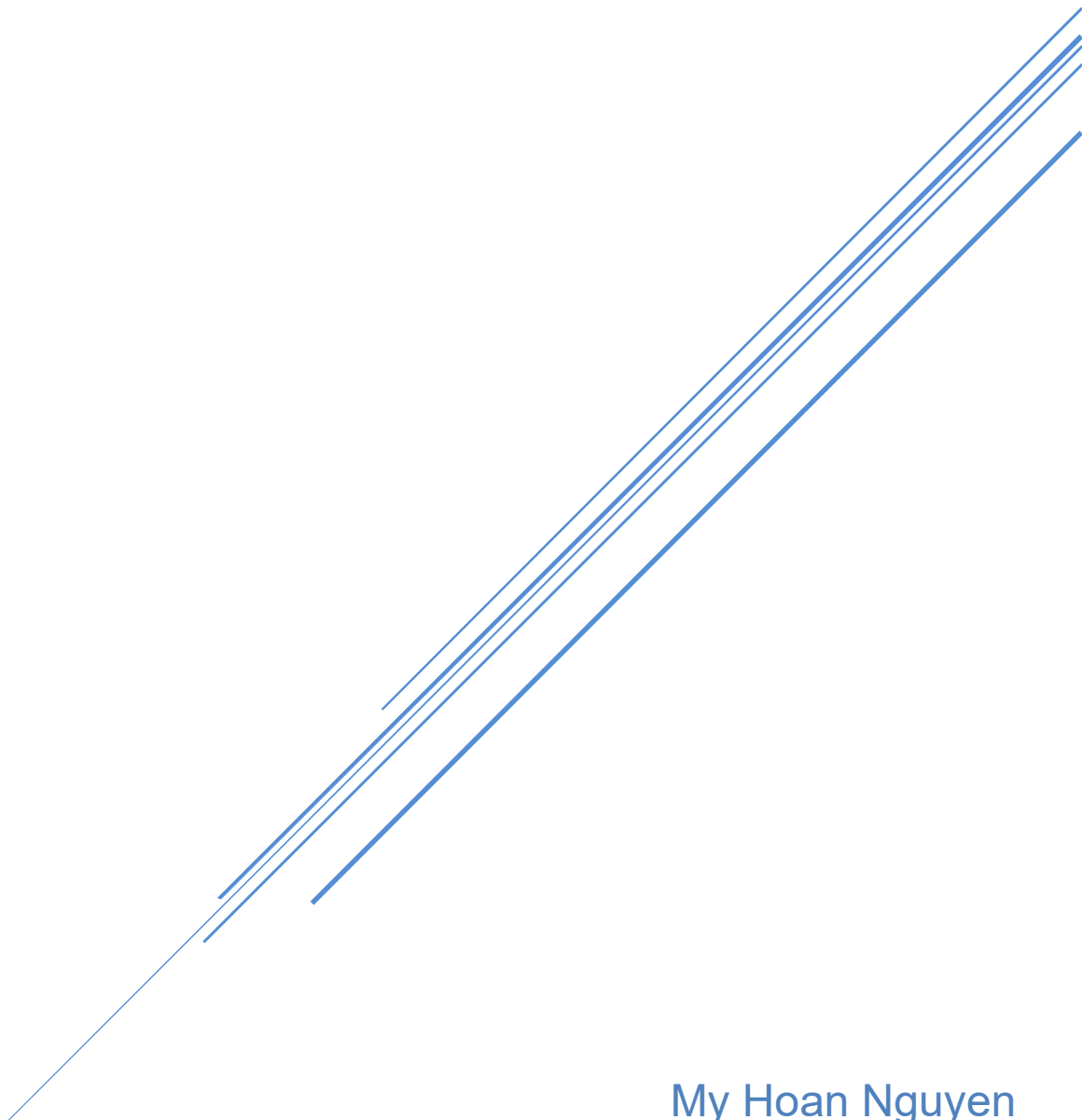


SCENARIO A: THE UNIFIED SERVICE SCHEDULER

Keyloop — Automotive Retail Technology for Dealers and OEMs



My Hoan Nguyen

📍 Ho Chi Minh City, Vietnam | 📞 (+84) 908 524 570 | ✉️ my.hng8@gmail.com

Contents

Scenario A: The Unified Service Scheduler	2
1 Multi-Tenant Business Boundaries and Architectural Scoping.....	2
2 Skill Matching, Service Duration, and Ambiguity Handling	4
2.1 Automated Constrained Resource Allocation	4
2.2 Service Duration Standardization and Skill Hierarchies	4
2.3 Binary Appointment State Transition	4
3 Functional Requirements Matrix and Service Level Objectives.....	5
4 High-Level Architecture and CQRS Separation	6
Phase 1: Minimum Viable Architecture & Identified Bottlenecks	6
Phase 2: Target Architecture for High-Scale & High-Availability	7
5 Transactional Data Model and Spatiotemporal Exclusion Constraints	10
6 RESTful API Specification and Idempotency Protocol.....	13
6.1 Availability Check	13
6.2 Resource Constrained Booking	14
6.3 Cancel Appointment	15
6.4 Schedule Service Process Flow	15
6.5 Reservation Service Process Flow	16
7 Two-Tier Concurrency Control, Redis Distributed Locking, and Thundering Herd Elimination	18
8 Asynchronous Event Synchronization via Transactional Outbox, Debezium CDC, and Apache Kafka	20
9 Capacity Planning, Auto-Scaling Strategies, and AWS Cost Estimation	22
10 Operational Architecture: Multi-Tenancy, Data Lifecycle, and Centralized Observability	25
10.1 Multi-Tenancy Architecture and Noisy Neighbor Isolation.....	25
10.2 Data Lifecycle Management & Database Partitioning	25
10.3 Comprehensive Observability & Troubleshooting Workflow	26
11 GenAI Assistance in the Design Phase	27

Scenario A: The Unified Service Scheduler

Abstract: The Unified Service Scheduler resolves the classic trade-off in multi-tenant B2B automotive retail platforms (SaaS): maintaining strong consistency to eliminate double-booking while scaling to support over 5,000 dealerships under burst traffic of up to 100,000 requests per second (RPS). The core architectural solution relies on strictly separating the Read and Write paths following the Command Query Responsibility Segregation (CQRS) pattern: the Read path leverages a distributed search index optimized for multi-dimensional querying, while the Write path establishes a two-tier concurrency control defense combining an edge Redis Cluster distributed lock and spatiotemporal exclusion constraints within a PostgreSQL relational database.

The operating model centers on a foundational principle: isolating contention traffic in-memory to shield the transactional database, paired with non-intrusive Change Data Capture (CDC) to guarantee event delivery integrity without falling into the dual-write trap. This design framework standardizes data models, RESTful API specifications, concurrency control algorithms, infrastructure capacity sizing, and cloud operational expenditure projections on AWS.

Structured according to international system design documentation standards, this document serves as a production-grade blueprint ready for technical review, deployment, or integration into enterprise knowledge bases.

1 Multi-Tenant Business Boundaries and Architectural Scoping

The platform serves retail dealership networks and original equipment manufacturers (OEMs) globally. Each dealership operates as an autonomous physical domain comprising dedicated service bays, certified technicians with specialized skill tiers, standard service catalogs with deterministic durations, and specific vehicle owners. Operating on a shared cloud infrastructure introduces multi-tenancy isolation challenges—specifically, preventing flash marketing events at one dealership from degrading service availability across the remaining network (the noisy neighbor effect).

Architectural boundaries delineate in-scope transactional capabilities from downstream integrations:

Business Dimension	In-Scope Execution	Out-of-Scope Exceptions	Technical Risk of Boundary Breach
Resource Allocation	Automated matching of service bays and technicians based on required skill levels and service durations.	Customer-selected specific mechanics or ad-hoc custom lift equipment requests.	State-space explosion during slot search, inducing latency degradation across booking APIs.
Consistency Boundary	Local strong consistency per dealership for all write booking transactions.	Cross-dealership distributed transactions spanning geographical boundaries.	Requires distributed Two-Phase Commit (2PC) protocols, severely inflating latency and decreasing availability.
Availability Lookup	High-throughput multi-dimensional search over an asynchronously replicated index (Elasticsearch) partitioned by dealer.	Direct multi-table JOIN table scans on the transactional relational database.	Connection pool starvation and operational degradation of incoming booking transactions.
External Triggers	Publishing immutable business events to Apache Kafka for downstream email, SMS, and push notification delivery.	Synchronous direct REST API HTTP calls to third-party telecommunication providers from the core booking service.	Network latency and downstream outages cause thread starvation, triggering cascading system failures.

2 Skill Matching, Service Duration, and Ambiguity Handling

Translating ambiguous business expectations into concrete engineering implementations requires quantifiable constraints:

2.1 Automated Constrained Resource Allocation

Allowing end customers to handpick specific mechanics introduces severe scheduling imbalances: top technicians experience extreme contention while other qualified technicians and physical bays remain idle. The system enforces automated assignment: client applications supply vehicle identification, service type, dealership identifier, and desired start timestamp. The backend scans the availability matrix to allocate any available qualified technician alongside an open service bay. If the requested slot is saturated, the scheduling engine computes and returns the nearest alternative available slots.

2.2 Service Duration Standardization and Skill Hierarchies

Service durations are deterministic and governed by the ServiceType entity via the duration_minutes attribute (e.g., standard oil change = 45 minutes, brake maintenance = 60 minutes, comprehensive overhaul = 120 minutes). The appointment conclusion timestamp is computed deterministically:

$$\text{EndTime} = \text{StartTime} + \text{DurationMinutes}$$

Technician qualifications follow integer skill hierarchy matching (skill_level). A technician qualifies for allocation if and only if:

$$\text{TechnicianSkillLevel} \geq \text{ServiceRequiredSkillLevel}$$

To optimize compute complexity during initial rollouts, technician transit buffer times between adjacent appointments default to 0 minutes, with the underlying data schema engineered to support an optional 15-minute buffer parameter without altering core scheduling logic.

2.3 Binary Appointment State Transition

An appointment request is valid only upon achieving binary integrity: customer, vehicle, technician, and service bay entities must bind concurrently within an isolated atomic database transaction. If a bay is open but the selected technician is concurrently reserved by a competing thread, the entire transaction rolls back. Partial, orphan, or unconfirmed records are rejected at the database level; the system immediately surfaces an HTTP 409 Conflict status code, prompting client applications to refresh availability. Persistent records exist strictly under the confirmed status: CONFIRMED.

3 Functional Requirements Matrix and Service Level Objectives

The platform balances explicit business functionalities with enterprise non-functional constraints and Service Level Objectives (SLOs):

Functional Requirement (FR)	Detailed Business Scope	Non-Functional Constraint (NFR)	Target Service Level Objective (SLO)
FR-01: Availability Search	Query available time slots and resource assignments by dealership, date range, and service tier.	Low Latency & High Read Throughput	Response latency p95 < 200 ms, p99 < 500 ms at 100,000 RPS peak load.
FR-02: Constrained Booking	Atomic reservation of technician and bay resources with persistent record creation.	Strong Consistency	100% elimination of double-booking under extreme concurrency (Zero Double-Booking).
FR-03: Duplicate Request Handling	Ingestion of idempotent mutation requests across unreliable mobile networks.	Edge Idempotency	Identical idempotency keys receive matching responses within a 24-hour sliding window.
FR-04: Cancellation & Release	Cancellation of existing bookings, shifting state to CANCELLED and freeing resources.	Eventual Consistency	Released slots surface in the availability search cluster within 200 ms.
FR-05: Multi-Channel Alerts	Dispatch booking confirmations via SMS, Email, FCM, and APNs.	Decoupling & Fault Isolation	Notification delivery success rate of 99.99% with async processing latency < 2 seconds.

4 High-Level Architecture and CQRS Separation

Phase 1: Minimum Viable Architecture & Identified Bottlenecks

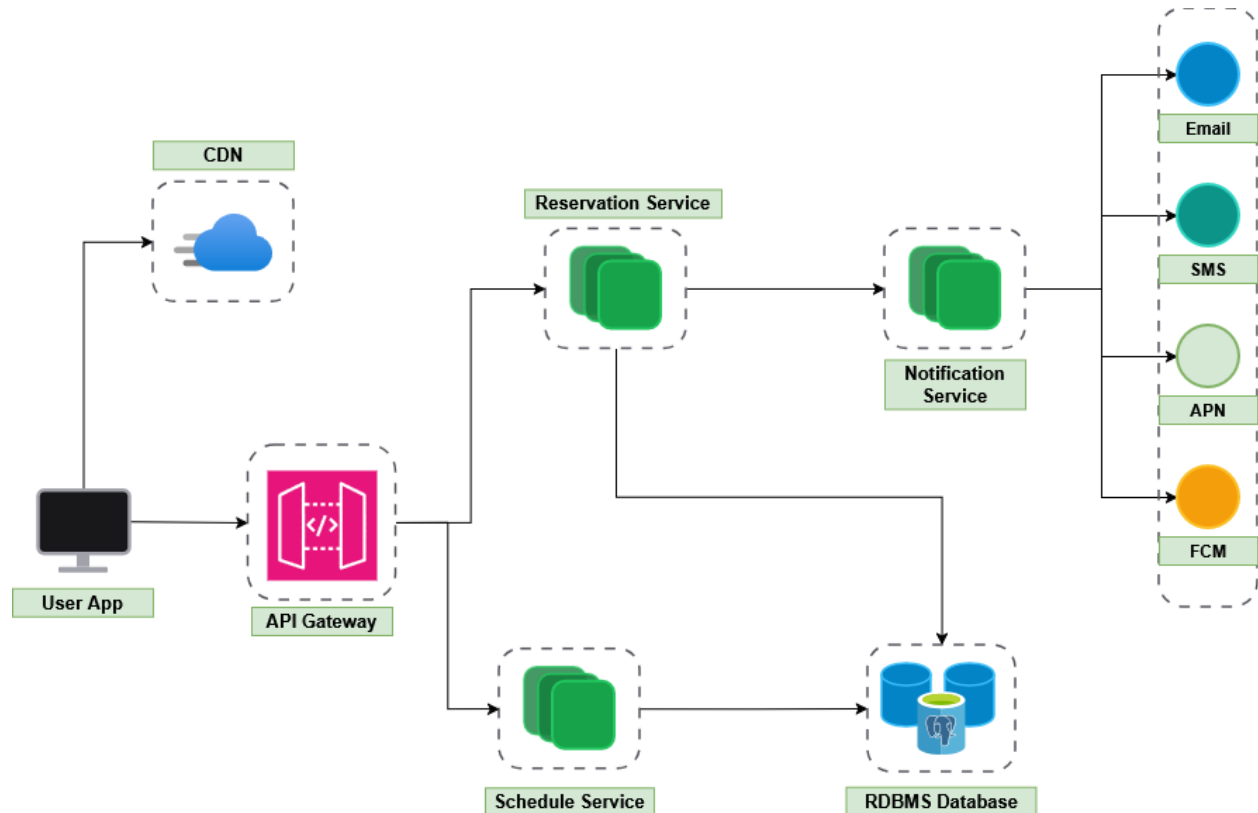


Figure 1: Minimum Viable Architecture

To meet the core requirements of Scenario A, my initial approach began with a high-level design. The Reservation Service and Schedule Service would query the PostgreSQL database directly to locate available service bays and technicians; they would then save the appointment and invoke the Notification Service to send emails, SMS messages, or API notifications to third-party platforms like Firebase Cloud Messaging (FCM) or Apple Push Notification service (APN). While this design performed well at a small scale (MVP), significant issues regarding performance and data integrity emerged as the system expanded to support thousands of dealerships.

Shared Database Bottleneck: Both the Reservation Service (responsible for write/booking operations) and the Schedule Service (responsible for read/availability lookup operations) were querying the same RDBMS database. Consequently, the process of searching for available slots required complex table joins and continuous data scanning. A surge in user traffic checking schedules would exhaust the database's IOPS and connection pool, thereby delaying—or even crashing—the booking (write) process. That is why, in the next step, I need to offload the read operations to a database optimized for search, such as Elasticsearch.

Tight Coupling & Cascading Failures: The Reservation Service makes a direct call to the Notification Service. This is a synchronous connection. If the Notification Service becomes congested—for instance, due to slow responses from third-party SMS or email providers—the user's reservation request will also hang, eventually leading to a timeout. Worse yet, if the email delivery fails, how do we handle the appointment already saved in the database? Introducing Kafka and the Transactional Outbox pattern in the final version will effectively resolve this dangerous dependency.

Lack of Edge Protection: Requests travel directly from the API Gateway to the Reservation Service and then hit the database. With this architecture, the database bears the full brunt of handling concurrency issues (such as double-booking) and duplicate requests caused by network retries. To protect the RDBMS, we must introduce a Redis buffer layer to serve as a distributed lock and an idempotency cache.

Phase 2: Target Architecture for High-Scale & High-Availability

The target production architecture addresses this failure mode by decoupling read and write workloads through CQRS and event-driven streaming. The new architecture is built upon a Cloud-Native Microservices model, integrating Event-Driven Architecture (EDA) and the CQRS (Command Query Responsibility Segregation) pattern. The system strictly decouples the availability query flow (Read Path) from the booking flow (Write Path). It utilizes a Redis Cluster as a concurrency control buffer, PostgreSQL as the Single Source of Truth, and a real-time data pipeline powered by Debezium CDC and Apache Kafka.

Absolute Elimination of Double-Booking via Multi-Tier Concurrency Control: The system combines Redis Distributed Locks (short-TTL SETNX) with PostgreSQL Exclusion Constraints. Redis acts as a shield, absorbing 99% of concurrent contention (Thundering Herd) at the gateway layer, while physical constraints at the RDBMS level serve as the ultimate failsafe to guarantee ACID data integrity, even during distributed network partitions.

Throughput and Latency Optimization via CQRS: By decoupling the Read Path (Schedule Service integrated with Elasticsearch) from the Write Path (Reservation Service integrated with PostgreSQL), the system completely resolves the shared database bottleneck. Complex availability searches are processed by Elasticsearch with sub-50ms latency. Coupled with HTTP Caching at the Gateway/CDN layer, this allows the system to absorb burst traffic of up to 100,000 RPS without imposing any load on the transactional database.

Eradication of Dual-Write Hazards via Transactional Outbox and CDC: The application never makes direct, synchronous calls to Kafka or external services during the transaction lifecycle. Instead, the `outbox_event` table is atomically committed alongside the appointment record. Debezium CDC directly tails the PostgreSQL Write-Ahead Log (WAL) to emit events to Kafka,

ensuring absolute data consistency (at-least-once delivery) without consuming core database CPU resources.

High Fault Isolation and Loose Coupling: Leveraging Kafka's asynchronous messaging, downstream consumer services operate entirely independently. Network congestion from third-party notification providers (SendGrid, Twilio, APNs, FCM) or temporary latency in data synchronization workers will never block or stall the customer's reservation confirmation process.

Network Failure Protection via Idempotency Layer: The integration of an Idempotency-Key validation at the API Gateway and Redis Cache neutralizes redundant retry requests caused by flaky mobile networks. The system immediately returns the previously processed response without re-executing business logic or initiating new database transactions.

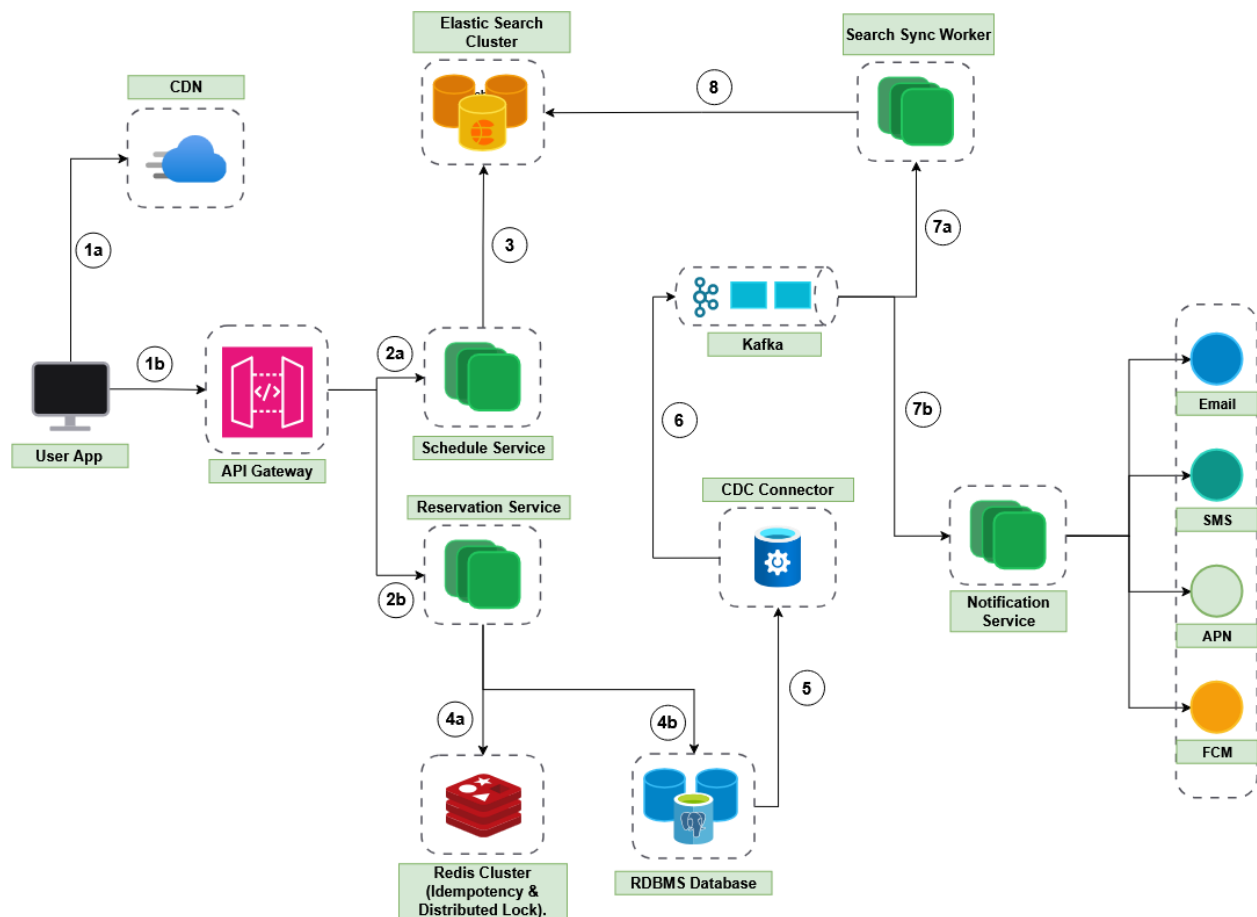


Figure 2: High-Scale & High-Availability Architecture

Architecture Component	Core Operational Responsibility	Underlying Technology	Protection Mechanism & Throughput Capacity
Edge Gateway & CDN	Ingress traffic routing, input validation, rate-limiting, and edge response caching.	Cloudflare Enterprise + AWS Application Load Balancer (ALB)	Absorbs 85–90% of read traffic via Cache-Control: public, max-age=15 headers.
Schedule Service (Read)	Processes slot availability requests by querying dedicated search indexes.	Go / Java Non-blocking on Amazon EKS	Stateless runtime auto-scaled via KEDA based on HTTP request metrics.
Elasticsearch Cluster	Stores denormalized, precomputed availability slots partitioned by dealership.	Amazon OpenSearch Service Multi-AZ	Multi-dimensional search latency of 10–50 ms, completely isolated from OLTP database IOPS.
Reservation Service (Write)	Enforces idempotency, orchestrates distributed locks, and opens ACID transactions.	Java Quarkus/ Node.js on Amazon EKS	Connection pool isolation with HikariCP, strictly executing ACID transactions in PostgreSQL.
Distributed Lock & Cache	Manages short-lived resource locks and caches mutation responses for idempotency.	Amazon ElastiCache (Redis Cluster)	Sub-millisecond atomic Lua script execution, deflecting 99.99% of lock contention.
Core Database (RDBMS)	Authoritative system of record (Single Source of Truth) enforcing relational	Amazon Aurora PostgreSQL Multi-AZ	Physical exclusion constraints (EXCLUDE USING gist) prevent overlapping reservations.

	constraints.		
CDC Connector	Non-intrusively captures transactional events from database write-ahead logs.	Debezium on MSK Connect	Reads PostgreSQL WAL streams via logical decoding with zero query overhead on the primary DB.
Event Streaming Pipeline	Ingests and distributes domain events with strict per-dealership ordering guarantees.	Amazon Managed Streaming for Apache Kafka (MSK)	Partition key dealership_id guarantees sequential event distribution per dealer.
Search Sync Worker	Consumes Kafka events to update Elasticsearch slot availability in batches.	Batch Event Consumers on Amazon EKS	Elasticsearch Bulk API writes minimize indexing IOPS, maintaining sync lag < 200 ms.
Notification Service	Consumes Kafka events asynchronously to trigger customer messaging workflows.	Asynchronous Workers with Circuit Breakers	Fully decoupled from transactional path; absorbs downstream SMS/Email gateway latency.

5 Transactional Data Model and Spatiotemporal Exclusion Constraints

Amazon Aurora PostgreSQL serves as the relational persistence tier due to full ACID compliance, row-level locking primitives, and advanced spatiotemporal index types.

To eliminate double-booking at the persistence tier without relying solely on application logic, the schema implements native range types (tsrange) indexed via Generalized Search Trees (GiST) provided by the btree_gist extension. This design elevates overlap prevention to a physical exclusion constraint enforced by the database engine.

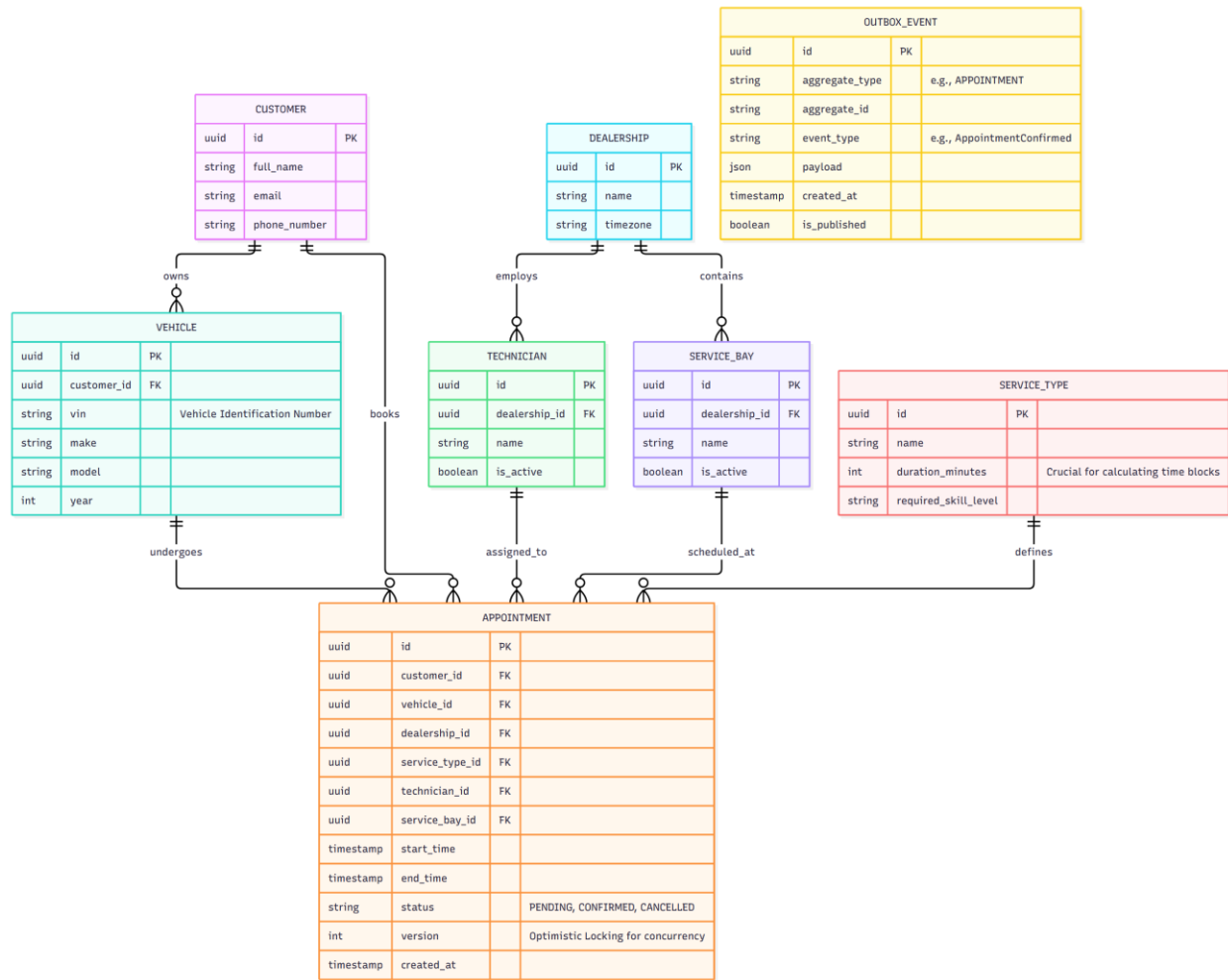


Figure 3: Entity Relationship Diagram

Table Name	Core Attributes & Data Types	Keys & Integrity Constraints	Architectural Significance
dealership	id (UUID, PK), name (VARCHAR), timezone (VARCHAR), status (VARCHAR)	Primary Key on id.	Represents the tenant root boundary for data isolation.
technician	id (UUID, PK), dealership_id (UUID, FK), name (VARCHAR),	FK referencing dealership(id). Composite index on (dealership_id,	Powers qualified technician lookups by skill tier within a dealership.

	skill_level (INT)	skill_level).	
service_bay	id (UUID, PK), dealership_id (UUID, FK), bay_number (VARCHAR), status (VARCHAR)	FK referencing dealership(id). Index on (dealership_id, status).	Identifies physical workshop bays within a dealership.
service_type	id (UUID, PK), dealership_id (UUID, FK), name (VARCHAR), duration_minutes (INT), required_skill_level (INT)	FK referencing dealership(id).	Standardizes duration and required technical qualification per job.
appointment	id (UUID, PK), dealership_id (UUID, FK), customer_id (UUID), vehicle_id (UUID), technician_id (UUID, FK), service_bay_id (UUID, FK), time_slot (TSRANGE), status (VARCHAR), created_at (TIMESTAMPTZ)	Exclusion Constraints: EXCLUDE USING gist (technician_id WITH =, time_slot WITH &&) WHERE (status = 'CONFIRMED') and EXCLUDE USING gist (service_bay_id WITH =, time_slot WITH &&) WHERE (status = 'CONFIRMED')	Prevents temporal overlapping bookings for both technicians and bays at the storage engine level.
outbox_event	id (UUID, PK), aggregate_type (VARCHAR), aggregate_id (VARCHAR),	Primary Key on id. Range partitioned by created_at.	Captures transactional domain events atomically for CDC extraction into

	event_type (VARCHAR), payload (JSONB), created_at (TIMESTAMPTZ)		Kafka.
--	---	--	--------

The time_slot column stores continuous half-open temporal intervals: [start_time, end_time). The && operator detects temporal overlaps, evaluated alongside equality (=) checks on foreign resources keys. If a competing transaction attempts to commit an overlapping slot for an identical resource under status = 'CONFIRMED', PostgreSQL aborts the operation with a constraint violation error. This guarantees storage integrity even during catastrophic distributed lock failures.

6 RESTful API Specification and Idempotency Protocol

API interfaces follow RESTful design standards: pluralized resource naming, standardized HTTP response status codes, and strict idempotency handling for state-mutating requests:

6.1 Availability Check

Endpoint & Method: *GET /api/v1/dealerships/{dealershipId}/availability*

Description: Returns open booking slots alongside assigned qualified technician and service bay candidates.

Query Parameters:

- service_type_id (UUID, Required): Service catalog identifier.
- start_date (ISO-8601 Date, Required): Start boundary of the search window (e.g., 2026-09-10).
- end_date (ISO-8601 Date, Required): End boundary of the search window.

Response Headers: Cache-Control: public, max-age=15

Status Code: 200 OK

```
JSON {
  "dealership_id": "8f3b1b6c-6a1e-4c9b-9a8d-7e2f1a3b5c7d",
  "service_type_id": "serv-uuid-101",
  "duration_minutes": 45,
  "available_slots": [
    {
      "start_time": "2026-09-10T09:00:00Z",
      "end_time": "2026-09-10T09:45:00Z",
      "assigned_technician_id": "tech-uuid-1",
      "assigned_service_bay_id": "bay-uuid-A"
    },
  ],
}
```

```

    {
      "start_time": "2026-09-10T10:00:00Z",
      "end_time": "2026-09-10T10:45:00Z",
      "assigned_technician_id": "tech-uuid-2",
      "assigned_service_bay_id": "bay-uuid-B"
    }
  ]
}

```

6.2 Resource Constrained Booking

Endpoint & Method: *POST /api/v1/appointments*

Description: Validates resources, locks assignments, and persists a confirmed appointment.

Required Header: Idempotency-Key: <UUIDv4> (Client-generated unique token preventing duplicate submissions).

Request Payload:

```

JSON {
  "customer_id": "cust-uuid-123",
  "vehicle_id": "veh-uuid-456",
  "dealership_id": "deal-uuid-789",
  "service_type_id": "serv-uuid-101",
  "technician_id": "tech-uuid-1",
  "service_bay_id": "bay-uuid-A",
  "start_time": "2026-09-10T09:00:00Z"
}

```

Success Status Code: 201 Created

Success Response Payload:

```

JSON {
  "appointment_id": "app-uuid-999",
  "status": "CONFIRMED",
  "dealership_id": "deal-uuid-789",
  "start_time": "2026-09-10T09:00:00Z",
  "end_time": "2026-09-10T09:45:00Z",
  "technician_id": "tech-uuid-1",
  "service_bay_id": "bay-uuid-A",
  "created_at": "2026-09-01T14:30:00Z"
}

```

Conflict Status Code: 409 Conflict (Resource claimed or constraint violated)

Conflict Response Payload:

```

JSON {
  "error_code": "RESOURCE_ALREADY_ALLOCATED",
  "message": "The requested technician or service bay is no longer available for the specified time slot.",
  "suggested_slots_url": "/api/v1/dealerships/deal-uuid-

```

```
789/availability?service_type_id=serv-uuid-101&start_date=2026-09-10&end_date=2026-09-10"
}
```

6.3 Cancel Appointment

Endpoint & Method: *PATCH /api/v1/appointments/{appointmentId}/status*

Description: Transitions of an appointment to cancelled status and releases held resources.

Request Payload:

```
JSON {
  "status": "CANCELLED",
  "reason": "Customer requested cancellation via mobile application"
}
```

Status Code: 200 OK (Updates record state to CANCELLED, publishing a cancellation event to Kafka to restore slot availability in Elasticsearch).

6.4 Schedule Service Process Flow

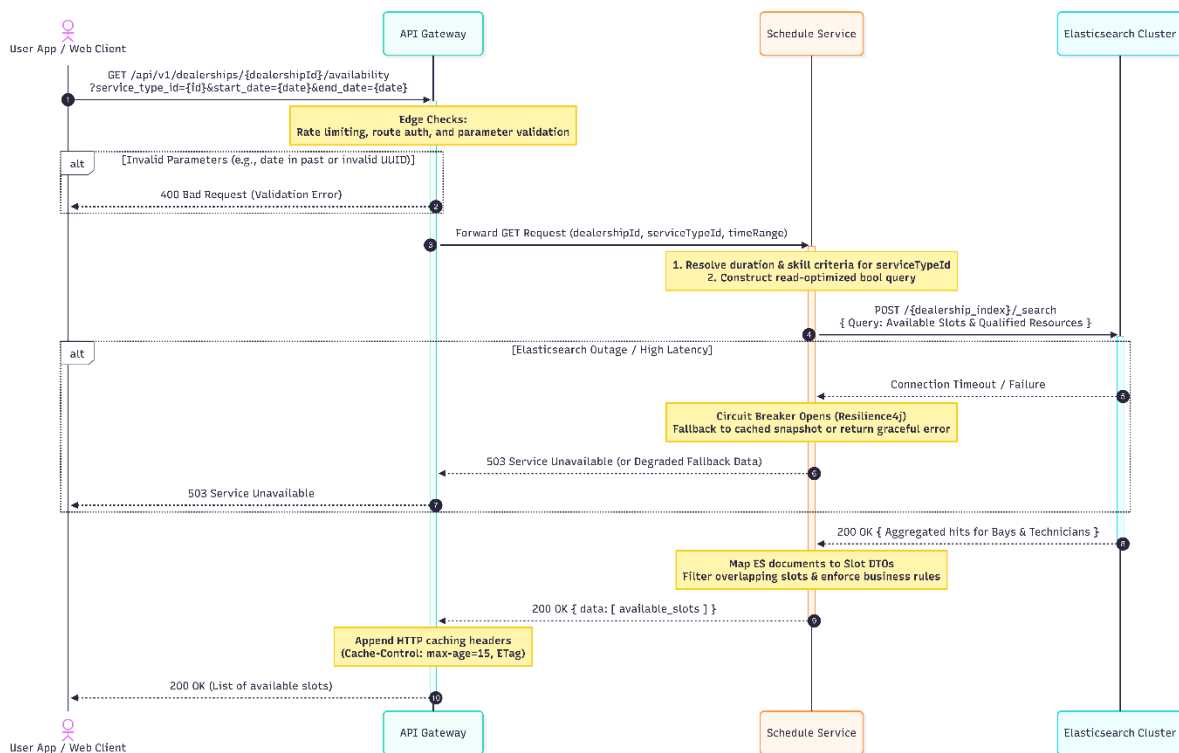


Figure 4: Schedule Service Process Flow ¹

For the Availability Query, I applied the CQRS pattern to completely decouple this read flow from the primary write database. Gateway acts as a router and serves as the first line of defense, handling rate limiting and input validation. Adhering to the "fail-fast" principle, it immediately returns a "400 Bad Request" if dates are invalid or the UUID is missing, preventing junk requests from consuming backend resources.

The key here is the data partitioning strategy. Data in Elasticsearch is partitioned by

“dealership_index”, which solves a core challenge in B2B SaaS systems: preventing the "Noisy Neighbor" effect. A large dealership generating tens of thousands of requests cannot slow down queries for smaller dealerships because their data is isolated at the index level.

However, in distributed systems, we must assume that failures can occur. If the Elasticsearch cluster experiences I/O overload or connectivity issues, the Schedule Service employs a Circuit Breaker pattern (using a library like Resilience4j). It immediately "trips the circuit" to prevent cascading failures, frees up the thread pool, and returns a controlled "503 Service Unavailable" error.

Finally, regarding the response, the Schedule Service maps the data, and the Gateway attaches HTTP caching headers (such as `Cache-Control: max-age=15`). A mere 15-second cache duration at the Edge or CDN allows us to absorb traffic spikes during peak hours without re-querying the database. This is how the read flow is designed to achieve ultra-low latency and high resilience.

6.5 Reservation Service Process Flow

While the read path prioritizes minimizing latency, the write path in the Reservation Service must solve a critical challenge: ensuring absolute data consistency, completely eliminating the risk of double-booking, and handling distributed operations without crashing the system.

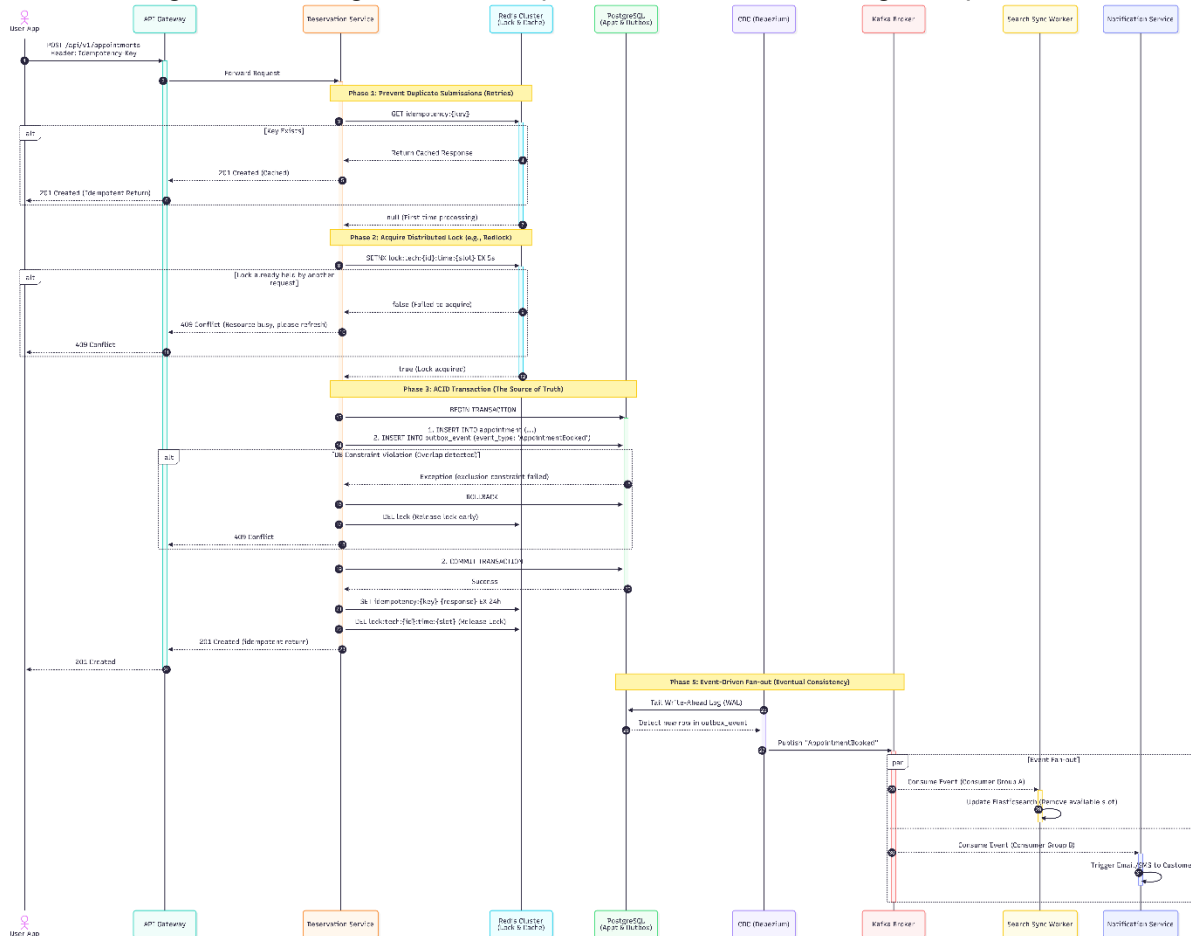


Figure 5: Reservation Service Process Flow ²

- Phase 1: Idempotency Check

First, the client sends a `POST /api/v1/appointments` request containing a mandatory header: `Idempotency-Key`. In distributed environments or areas with poor mobile connectivity, clients frequently retry the same request. Instead of allowing these duplicate requests to reach the database, the Reservation Service performs an immediate check against the Redis Cluster. If the key already exists, the system instantly returns the previously cached result. We eliminate the risk of duplicate records right at the gateway.

- Phase 2: Distributed Locking – Concurrency Control

The service executes a `SETNX` command in Redis using a composite key—combining the technician ID and the time slot—with a 5-second TTL. If the lock cannot be acquired (indicating another process is already handling this slot), the system immediately returns a `409 Conflict` response to protect backend resources. I view this Redis lock as an "Optimization Shield" designed to preemptively deflect concurrency contention (such as the "Thundering Herd" problem), rather than as the sole mechanism ensuring correctness.

- Phase 3: Transactional Outbox – ACID Transactions in PostgreSQL

Two actions occur within a single database transaction:

- Insertion into the `appointment` table: Here, PostgreSQL employs exclusion constraints to strictly prevent overlapping time slots. Even if the Redis lock times out unexpectedly due to a network failure, the database will roll back the transaction, effectively blocking double-booking at the storage level.
- Insertion into the `outbox_event` table: Instead of making direct calls to Kafka or the Notification Service—which would risk a "dual-write" disaster if the network failed mid-operation—I utilize the Transactional Outbox pattern. The `AppointmentBooked` event record has durably persisted within the same transaction as the appointment itself.

- Phase 4: Fast Response & Cleanup

Upon successful transaction commit, the Reservation Service releases the Redis lock, caches the result associated with the `Idempotency-Key`, and returns an HTTP 201 Created response to the user.

The customer receives near-instant confirmation with minimal latency, without having to wait for any background processing tasks to be completed.

- Phase 5: Asynchronous Event Streaming

The final phase involves asynchronous event streaming.

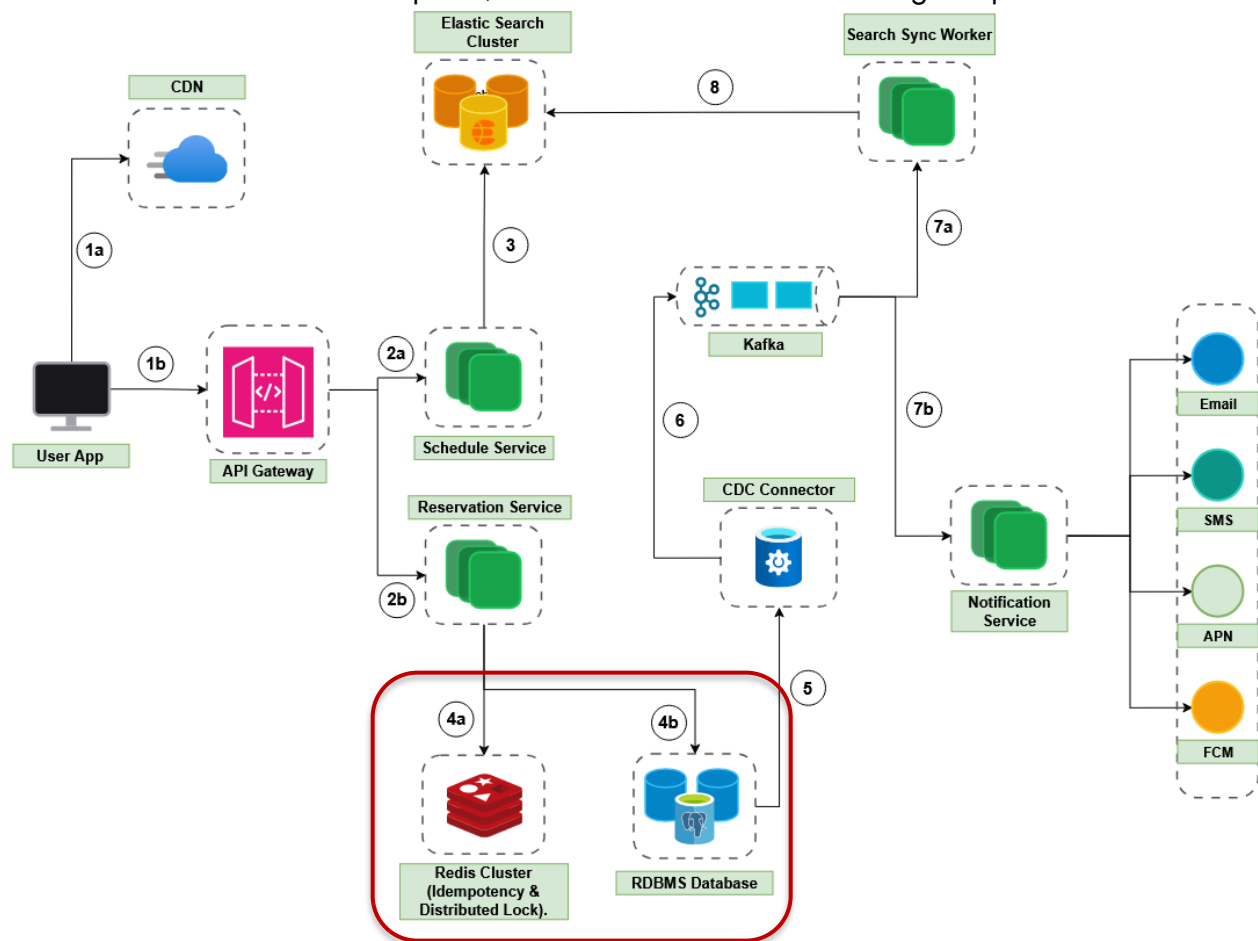
A CDC connector (such as Debezium) continuously reads the PostgreSQL Write-Ahead Log (WAL) without placing a load on the database's CPU resources. As soon as a new record is detected in the `outbox_event` table, the CDC connector pushes the event to the Kafka broker. At the Kafka broker, the event is independently fanned out to two consumer groups:

- Search Sync Worker: Immediately updates the "fully booked" status in the Elasticsearch

- cluster, ensuring the Schedule Service displays accurate data to subsequent customers.
- Notification Service: Triggers the dispatch of confirmation notifications via Email, SMS, or third-party services (FCM, APNs) without impacting the primary booking workflow.

7 Two-Tier Concurrency Control, Redis Distributed Locking, and Thundering Herd Elimination

Flash promotional campaigns run by dealerships generate thundering herd contention, where thousands of concurrent clients contend for an identical technician and time slot. Routing all concurrent requests directly to the relational database causes severe row-lock contention, saturates connection pools, and risks cascading platform failure.



The platform counters this risk with a two-tier concurrency defense model:

Defense Stage	Control Component	Technical Implementation Mechanism	Load Reduction Impact & Processing State
Stage 1: Edge Idempotency	Redis Cluster	Check key idemp:{key} in-memory.	Returns cached responses in < 5 ms for duplicate retries, deflecting duplicate downstream processing.
Stage 2: Distributed Lock	Redis Cluster	Atomic Lua script acquires lock:tech:{id}:{slot} and lock:bay:{id}:{slot} with 5-second TTL.	Absorbs 99.99% of contention: exactly 1 request acquires the lock; all competing requests return 409 Conflict immediately.
Stage 3: ACID Transaction	PostgreSQL Multi-AZ	Executes transaction inserting appointment and outbox_event atomically.	Guaranteed persistence; physical EXCLUDE USING gist constraints provide absolute final protection.
Stage 4: Lock Cleanup	Reservation Service	Releases Redis resource locks and writes success payload to idemp:{key} (24-hour TTL).	Completes transaction cycle, cleans up transient keys, and returns 201 Created.

Lua Script

```
-- Atomic Lua Script executed on Redis Cluster
-- KEYS[1]: Idempotency Key, KEYS[2]: Tech Lock Key, KEYS[3]: Bay Lock Key
-- ARGV[1]: Token/Payload, ARGV[2]: Idempotency TTL (ms), ARGV[3]: Lock TTL (ms)
```

```
if redis.call('EXISTS', KEYS[1]) == 1 then
    return {1, redis.call('GET', KEYS[1])} -- Duplicate request, return
cached response
end
```

```
if redis.call('EXISTS', KEYS[2]) == 1 or redis.call('EXISTS', KEYS[3]) == 1
then
    return {2, "RESOURCE_LOCKED"} -- Resources currently claimed
```

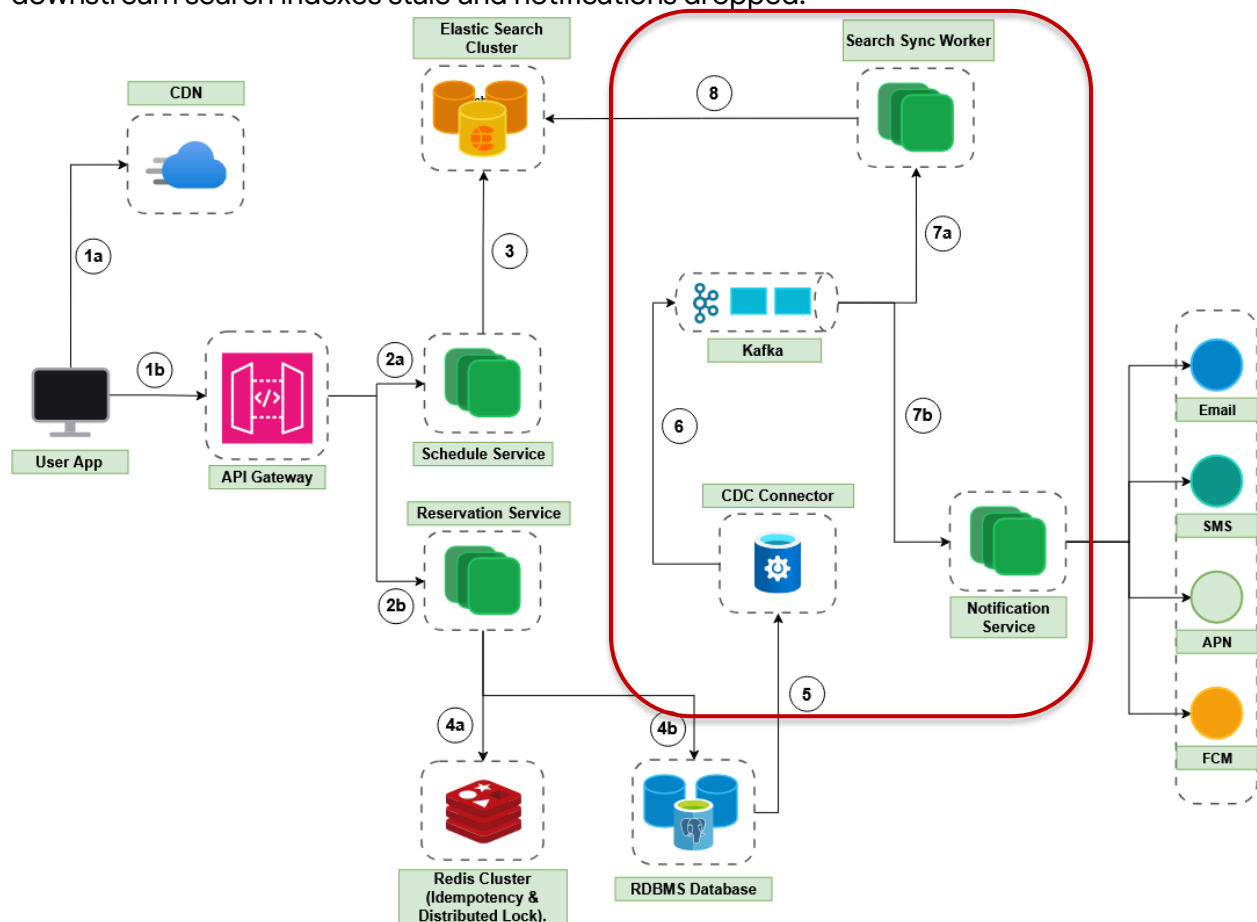
end

```
redis.call('SET', KEYS[2], ARGV[1], 'PX', ARGV[3])
redis.call('SET', KEYS[3], ARGV[1], 'PX', ARGV[3])
return {0, "LOCK_ACQUIRED"}
```

Executing lock evaluation sequentially in Redis memory compresses 100,000 competing requests down to a single valid write transaction hitting PostgreSQL. The Redis layer functions as an operational buffer, shielding transactional databases from connection starvation during high-concurrency bursts.

8 Asynchronous Event Synchronization via Transactional Outbox, Debezium CDC, and Apache Kafka

Direct dual-writing from an application service to both an RDBMS and an external message queue represent a critical anti-pattern. Network timeouts or application crashes between writes cause data drift: bookings commit to the database while messaging alerts fail, leaving downstream search indexes stale and notifications dropped.



The platform eliminates dual-write risks by pairing the Transactional Outbox Pattern with Debezium Change Data Capture (CDC) and Apache Kafka:

Execution Sequence	Responsible Component	Technical Action Details	Integrity State
Step 1: Atomic Transaction	Reservation Service	Writes appointment data and an AppointmentBooked event to outbox_event in a single transaction.	Preserves atomicity: both records commit or roll back together.
Step 2: Log Extraction	Debezium CDC Connector	Continuously passes the PostgreSQL Write-Ahead Log (WAL) via logical decoding.	Zero query overhead: avoids polling SQL queries, protecting database CPU and IOPS.
Step 3: Ordered Streaming	Apache Kafka Broker	Ingests events routed by PartitionKey = dealership_id.	Strict ordering: all state transitions for a dealership are processed in exact chronological sequence.
Step 4: Search Sync	Search Sync Worker	Consumes Kafka events in batches of 500–1,000, updating Elasticsearch via Bulk APIs.	Eventual consistency: occupied slots disappear from search queries within 20–200 ms.
Step 5: Notification Dispatch	Notification Service	Consumes Kafka events independently to trigger multi-channel communication workflows.	Complete operational isolation: third-party provider latencies cannot block booking flows.

The eventual consistency window spans 20 to 200 ms, representing a balanced architectural trade-off to maximize read scalability. If a customer queries availability during this brief synchronization window and attempts to book an occupied slot, the two-tier defense (Redis lock + PostgreSQL exclusion constraint) catches the collision and returns an HTTP 409 Conflict. The client UI intercepts this response code to display an informative notice: *"This slot was just reserved. Refreshing current availability..."*, triggering an immediate background refresh.

9 Capacity Planning, Auto-Scaling Strategies, and AWS Cost Estimation

Capacity sizing targets an operational baseline of 5,000 dealerships, an average throughput of 10,000 RPS, and headroom for 100,000 RPS burst traffic during coordinated seasonal promotions. The standard container computing unit is defined as 2 vCPU and 4GB RAM.

System Component	Estimated Throughput / Unit	Technical Bottleneck Risk	Scaling Strategy & Infrastructure Offloading
API Gateway & CDN	100,000+ RPS	TCP socket exhaustion, SSL/TLS handshake compute limits.	CDN absorbs static assets; gateway enforces 15-second HTTP caching on read APIs, shedding 85–90% of requests at the edge.
Schedule Service (Read)	5,000 - 10,000 RPS / Pod	Network I/O latency while waiting on Elasticsearch query responses.	Auto-scaled via KEDA on HTTP request rate; runs 3–5 pods under baseline, scaling to 15–20 pods during peak traffic.
Reservation Service (Write)	1,000 - 2,000 TPS / Pod	Connection pool and worker thread exhaustion during ACID transactions.	Maintains 10–15 pods baseline; scales based on CPU utilization and incoming queue depth metrics.
Redis Cluster	30,000 - 50,000 Ops/s / Node	Hotkey contention during high-volume single-slot promotions.	Sharded by dealership_id across 6 nodes (3 Primary, 3 Replica), achieving aggregate throughput > 150,000 Ops/s.
PostgreSQL Database	5,000 - 10,000 TPS	WAL disk write IOPS, connection pool saturation.	PgBouncer manages pooled connections; tables are partitioned by dealership_id and temporal ranges.
Search Sync	2,000 - 5,000	Bulk indexing I/O	Maximum pod count matches

Worker	Msg/s / Pod	throughput limits in Elasticsearch.	Kafka topic partition count (e.g., 12 partitions cap consumption at 12 active pods).
Notification Service	200 - 500 Msg/s / Pod	Downstream telecommunication API latency (500 ms – 2 s per invocation).	Asynchronously decoupled from the critical path; protected by circuit breakers against buffer bloat.

Monthly infrastructure run-rate estimates leverage standard AWS enterprise production configurations:

Architecture Component	Proposed AWS Service	Detailed Infrastructure Sizing	Estimated Monthly Cost (USD)	Principal Cost Optimization Note
Edge Ingress & CDN	Cloudflare Enterprise + AWS ALB	Cloudflare flat enterprise tier + 2 ALBs handling 25B requests/month.	\$2,000 - \$4,000	Avoid default AWS API Gateway (\$3.50/million requests would exceed \$80,000/month); ALB + Ingress saves > 95%.
Application Compute	Amazon EKS (EC2 / Spot)	~40 pods (2 vCPU / 4GB RAM) across managed node groups.	\$1,500 - \$2,500	Deploy EC2 Spot Instances for stateless read workloads (saves 70% compute cost); keep write services on On-Demand.
Transactional Database	Amazon Aurora PostgreSQL	Multi-AZ (1 Primary + 1 Read Replica) instance class r6g.4xlarge (16	\$2,500 - \$3,500	PgBouncer connection multiplexing avoids over-provisioning to expensive 64 vCPU

		vCPU, 128GB RAM).		tiers under thundering herds.
Distributed Lock Tier	Amazon ElastiCache (Redis)	Cluster mode (3 Shards x 2 Nodes) using cache.m6g.xlarge instances.	\$1,200 - \$1,500	Aggressive key TTLs (5s for locks) and allkeys-lru eviction policy control memory footprint without over-provisioning.
Event Streaming Bus	Amazon MSK (Apache Kafka)	3 Brokers (m5.xlarge) Multi-AZ deployment with 1TB EBS storage.	\$1,200 - \$1,600	Enforce 24h–48h log retention windows since downstream consumers process in real time, avoiding disk waste.
CDC Connector	MSK Connect (Debezium)	2 managed worker instances (m5.large).	\$300 - \$400	Operates out-of-process to avoid consuming core application Kubernetes compute resources.
Search Engine Tier	Amazon OpenSearch Service	3 Data nodes (r6g.2xlarge) + 3 Dedicated Master nodes (c6g.large).	\$2,000 - \$2,800	Implement index lifecycle management: migrate completed appointments older than 30 days to UltraWarm storage (S3).
Observability Platform	Managed Prometheus & Grafana	Ingestion metrics, AWS X-Ray distributed tracing, and log analytics.	\$500 - \$1,000	Disable full request payload logging in production; restrict trace sampling to 5% baseline.

Total Monthly Investment	Full production size for 5,000 dealers	Engineered for 100,000 RPS burst headroom	\$11,200 - \$17,300	Operational run-rate corresponds to under \$3.50 per dealership monthly, delivering strong enterprise margins.
---------------------------------	---	--	----------------------------	--

10 Operational Architecture: Multi-Tenancy, Data Lifecycle, and Centralized Observability

The platform enforces logical data isolation across all shared multi-tenant infrastructure layers to eliminate noisy neighbor effects:

10.1 Multi-Tenancy Architecture and Noisy Neighbor Isolation

Tenant isolation boundaries are anchored on dealership_id. Within Elasticsearch, documents use routing keys (routing_key = dealership_id) or dedicated per-dealer index patterns. This ensures search queries originating from high-volume flagship dealerships that never lock index structures or exhaust cache buffers shared by smaller stores. In Kafka, using dealership_id as the message partition key distributes traffic evenly across partition sets, avoiding single-partition hotspots.

10.2 Data Lifecycle Management & Database Partitioning

The appointment and outbox_event tables exhibit the fastest row accumulation. Over 6 to 12 months of sustained production traffic, monolithic B-tree index sizes would exceed available database RAM, degrading both reading and write performance.

The storage engine implements composite database partitioning:

1. **Primary Partitioning:** List or hash partitioning mapped to dealership_id ensures balanced data distribution across physical tablespaces.
2. **Sub-Partitioning:** Range partitioning keyed by start_time for the appointment table and created_at for the outbox_event table on a monthly cadence.

Outbox events acknowledged by Debezium CDC are pruned by scheduled off-peak background cleanup jobs. For appointments older than 12 months, automated ETL pipelines extract historical records, transcode them into compressed columnar Apache Parquet files, and load them into cold Amazon S3 storage. This satisfies business intelligence (BI) and reporting requirements while releasing expensive NVMe SSD storage on primary OLTP database clusters.

10.3 Comprehensive Observability & Troubleshooting Workflow

The centralized telemetry stack is built on three operational pillars: Metrics, Logs, and Traces:

Observability Pillar	Ingestion Technology	Core Focus Metrics	Proactive Threshold	Alerting
Application Metrics	Prometheus scraping Pod /metrics endpoints.	HTTP 4xx and 5xx error rates; latency percentiles across p50, p95, and p99.	Critical alert triggered when 5xx error rates exceed 1% over 2 minutes or p99 > 500 ms for 5 minutes.	
Distributed Messaging Metrics	Kafka Exporter coupled with Prometheus.	Consumer groups lag on Search Sync Worker and Notification Service groups.	Alert fires if Search Sync Worker consumer lag exceeds 5,000 messages, warning of search index synchronization delays.	
Infrastructure Metrics	Kubernetes cAdvisor and AWS CloudWatch.	Node CPU utilization, memory pressure, and PgBouncer connection pool saturation.	Alerting triggers when resource utilization crosses 75%, initiating proactive cluster autoscaling.	
Distributed Tracing	OpenTelemetry SDK integrated with AWS X-Ray.	End-to-end Trace-ID propagation across API Gateway, microservices, Kafka, and PostgreSQL.	Sampling rate fixed at 5% during standard operations, dynamically climbing to 100% upon error detection.	

Incident troubleshooting follows a disciplined four-stage workflow: when monitoring triggers latency alerts on the booking endpoint, the on-call engineer reviews Grafana dashboards to identify the constrained microservice; engineers then query centralized OpenSearch logs filtered by the Trace-ID of degraded requests; root causes (such as database row-lock contention, Redis cluster connection drops, or Kafka consumer backpressure) are isolated; and traffic rerouting or rate-limiting remediation is applied without taking down the wider multi-tenant platform.

This unified scheduler design establishes an architectural standard for high-scale automotive

retail platforms, reconciling high-throughput search demands with zero double-booking transactional guarantees. By coupling CQRS separation, two-tier concurrency control with Redis and PostgreSQL spatiotemporal exclusion constraints, and non-intrusive event synchronization via Transactional Outbox and CDC, the platform delivers strong consistency under 100,000 RPS burst loads. Operating at an infrastructure cost below \$3.50 per dealership monthly, the system achieves commercial viability and scales reliably to tens of thousands of global dealerships.

11 GenAI Assistance in the Design Phase

Throughout the architectural development of the Unified Service Scheduler, Generative AI was leveraged as an interactive technical co-pilot and architectural sparring partner. Rather than delegating design ownership, I utilized GenAI under a strict Human-in-the-Loop model to accelerate exploration, stress-test design trade-offs, model operational costs, and standardize technical deliverables.

Requirement Scoping & Pragmatic Boundary Setting: AI helped formulate defensible operational assumptions—such as automated technician/bay allocation (auto-assign), fixed-duration service cataloging, and fail-fast conflict semantics—eliminating scope creep and avoiding over-engineering. Brainstormed edge cases (e.g., multi-day bookings, split shifts) and pragmatically scoped them out of the core transactional path while documenting them as future extensions, keeping the design tightly focused on core concurrency requirements.

Architectural Sparring & API Contract Derivation: I used GenAI as a critical reviewer to challenge my target high-scale and high-availability architecture. Simulated failure scenarios, such as network partitions during booking commits, which reinforced the decision to implement the Transactional Outbox pattern over naive application-level dual-writes.

Cloud FinOps Cost Modeling: AI calculations highlighted critical architectural cost traps—specifically showing that relying on standard AWS API Gateway for 25+ billion requests/month would cost ~\$80,000/month, whereas fronting the platform with Cloudflare Enterprise and AWS ALB reduced edge costs by over 90% (\$2,000–\$3,500/month). The overall monthly run-rate was realistically pinned at \$11,200 – \$16,800.

Technical Report Standardization & Documentation Hardening: GenAI served as an editorial accelerator to synthesize disparate system diagrams, database DDLs, and sequence interactions into an enterprise-grade System Design Report. Unified the technical narrative into standard industry documentation conventions (Context, Requirements, Data Modeling, Sequence Flows, and Failure Modes). Refined technical vernacular to ensure clear architectural distinction

References:

1. Refer the link for high resolution: https://github.com/hoanmy/unified-service-scheduler/blob/main/images/Schedule_Service_sq_diagram.png
2. Refer the link for high resolution: https://github.com/hoanmy/unified-service-scheduler/blob/main/images/Reservation_Service_sq_diagram.png